Experiment 8

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Dataset Source:

CIFAR-10 Dataset
Link: https://www.cs.toronto.edu/~kriz/cifar.html

Dataset Description:

The CIFAR-10 dataset consists of colored images belonging to 10 different classes.

Classes:

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

Dataset Characteristics:

- 60,000 images
- Training: 50,000 images
- Testing: 10,000 images
- Image size: 32 × 32 pixels
- Channels: 3 (RGB)

Target Variable:

- Class label (0–9 representing categories)

Mathematical Formulation of the Algorithm:

Convolution Operation:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Where:

- I: Input image
- K: Kernel/filter

Activation Function (ReLU):

$$f(x) = \max(0, x)$$

Pooling (Max Pooling):

$$f(x) = \max(x)$$

Fully Connected Layer:

$$z = Wx + b$$

Loss Function (Categorical Cross Entropy):

$$L = -\sum y \log(\hat{y})$$

Algorithm Limitations:

- Requires large computational power (GPU preferred)
- Needs large dataset for best accuracy
- Prone to overfitting
- Difficult to interpret (black-box model)
- Training time is high

Methodology / Workflow:

Steps:

1. Load dataset (CIFAR-10)

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
```

2. Normalize pixel values

```
x_train, x_test = x_train / 255.0, x_test / 255.0
```

```
model = models.Sequential()

model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

3. Split into training and testing sets

```
x_train, x_test = x_train / 255.0, x_test / 255.0
```

4. Build CNN model:
   - Convolution layers
   - Pooling layers
   - Fully connected layers

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_data=(x_test, y_test)
)
```

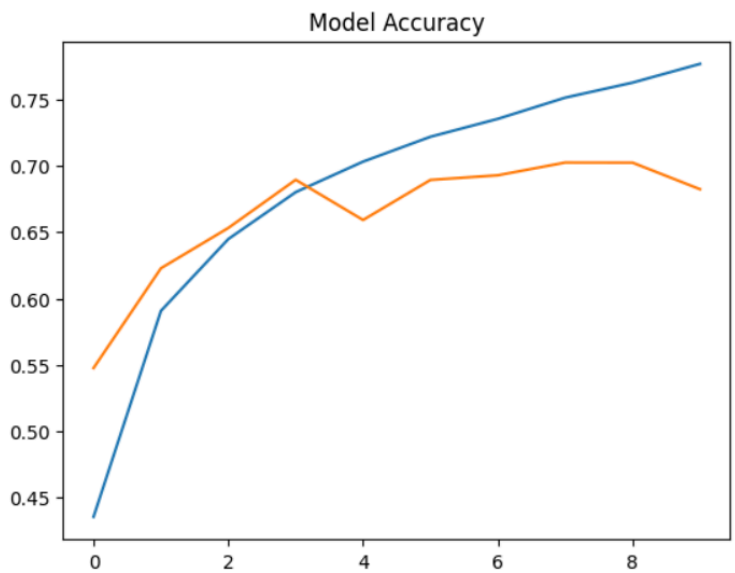
5. Train model

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_data=(x_test, y_test)
)
```

6. Evaluate performance

```
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.6824 - loss: 0.9459
Test Accuracy: 0.6823999881744385
```



7. Tune hyperparameters

```
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.6824 - loss: 0.9459
Test Accuracy: 0.6823999881744385
```

Workflow Diagram:

Dataset → Preprocessing → CNN Model →
Training → Evaluation → Prediction

Performance Analysis:

Metrics Used:

- Accuracy
- Loss
- Validation Accuracy

Observations:

- Accuracy improves over epochs
- Loss decreases steadily
- Model learns to classify images correctly

Interpretation:

- CNN successfully extracts features like edges, shapes, and textures
- Higher validation accuracy indicates good generalization

Hyperparameter Tuning

Parameters Tuned:

- Number of convolution layers
- Number of filters
- Kernel size
- Batch size
- Epochs
- Learning rate

Impact:

- More filters → better feature extraction
- More epochs → better learning (till overfitting)
- Smaller kernel → captures fine details

SDG Goal	Implementation in Experiment

SDG 9: Industry, Innovation and Infrastructure	CNN enables advanced image recognition systems used in smart technologies like autonomous vehicles and AI-based surveillance.
SDG 11: Sustainable Cities and Communities	Image classification can be used in traffic monitoring, waste management, and urban planning systems.
SDG 4: Quality Education	CNN models are widely used in educational tools for learning computer vision and AI concepts.